

# Computer Architecture — Building the CPU & the Computer

## Lab 5 (Capstone) — Nand2Tetris Project 5

Auqib Hamid Lone

*Department of Computer Science & Engineering, GCET Ganderbal*

August 21, 2026

---

**Aim.** *To integrate the ALU (Lab 2) and registers/PC (Lab 3) with instruction-decoding control logic into a complete Central Processing Unit, and to assemble the CPU, Memory and ROM into a full stored-program computer — the capstone of the hardware track.*

### 5.1 Learning Outcomes

On completing this lab, a student will be able to:

- Explain the fetch–decode–execute cycle as realised in hardware, not just in theory.
- Design control logic that decodes a 16-bit instruction into the ALU control bits and register load signals.
- Compose the CPU, Memory (RAM16K + Screen + Keyboard) and ROM32K into the top-level Computer chip.
- Reason about a von Neumann-style architecture end-to-end: instruction fetch, execution, and memory-mapped I/O.

### 5.2 Pre-Lab Reading

**Pre-Lab Reading.** This lab’s CPU-organization prerequisites span three CS401 weeks. **Week 5** introduced the CPU’s internal datapath — the register file feeding a common ALU across multiple internal buses (Mano, *Computer System Architecture*, 3rd ed., Fig. 8-2, p.243, “Registers with common ALU”; the same three-bus principle in Hamacher Ch. 7, chapter-level) — `CPU.hdl` is a concrete, minimal instance of exactly this: an ALU (Lab 2), an A-register and D-register (Lab 3’s **Register**), and a PC (Lab 3), wired together. **Week 6** introduced hardwired control-unit design: a step counter feeding a decoder that produces timed control signals from the current instruction (Mano, Fig. 5-6, p.137) — `CPU.hdl`’s instruction-decoding logic is the same idea specialised to Hack’s single-cycle (no multi-step sequence counter needed) instruction format. **Week 7** (microprogrammed control, Mano Ch. 7; Stallings Ch. 16, chapter-level) is useful contrast reading — Hack’s CPU is hardwired, not microprogrammed, and seeing why that choice is viable for a minimal ISA sharpens the Week 6/7 comparison from the lecture itself.

New in this lab, not covered by any CS401 lecture: the specific bit-to-control-signal mapping for the Hack C-instruction (re-derive which `comp` bits feed the ALU’s `zx/nx/zy/ny/f/no` vs. which

`dest/jump` bits drive register-load and PC-load logic) — see Nisan & Schocken, *The Elements of Computing Systems*, Ch. 5, *Computer Architecture*, and the CPU/Memory chip APIs supplied in the project scaffold.

## 5.3 Toolchain Setup

1. Install a Java Runtime Environment (JRE), version 8 or later, if not already present.
2. Download the Nand2Tetris software suite (free, [nand2tetris.org](http://nand2tetris.org)) — this lab uses the Hardware Simulator (for `CPU.hdl`/`Memory.hdl`/`Computer.hdl`) and the CPU Emulator (to re-run Lab 4's `Mult.asm`/`Fill.asm` on your own `Computer.hdl`).
3. Locate `projects/05/` — it contains `CPU.hdl`, `Memory.hdl`, and `Computer.hdl` stubs plus their supplied `.tst/.cmp` scripts, and reuses every chip built in Labs 1-4 (place copies of, or path-reference, your prior labs' `.hdl` files as the toolchain's built-in-chips search path requires).
4. Load each `.tst` script in the Hardware Simulator and run it; a correct chip prints `End of script --- comparison ended successfully`.
5. To demonstrate the full computer, load `Computer.hdl` in the Hardware Simulator, load one of Lab 4's compiled `.hack` programs (`Mult.hack` or `Fill.hack`) into its ROM32K, and run — confirm `RAM[2]` ends up correct for `Mult`, and that `Fill`'s screen behaviour is unchanged from Lab 4's CPU-Emulator demo, now running on hardware you built yourself rather than the reference implementation.

## 5.4 Tasks

1. `CPU(inM[16],instruction[16],reset) → (outM[16],writeM,addressM[15],pc[15])` — implement by wiring together an ALU (Lab 2), an A-register and a D-register (Lab 3's `Register`), and a PC (Lab 3), with instruction-decoding logic built from Lab 1 gates:
  - (a) If `instruction[15]=0` (A-instruction), load the A-register with `instruction` directly; do not touch the ALU or D-register.
  - (b) If `instruction[15]=1` (C-instruction), route `instruction[6]` (the `a` bit) to select the ALU's `y` input as `A` or `inM` (`M`); route `instruction[11..6]` (the `comp` bits) directly as the ALU's `zx,nx,zy,ny,f,no` control bits (this is Lab 2's ALU control-bit table, reused verbatim, not reinvented); use `instruction[5..3]` (the `dest` bits) to gate whether the ALU's result loads into `A`, `D`, and/or is written to `outM` (`writeM`); use `instruction[2..0]` (the `jump` bits) together with the ALU's `zr/ng` outputs to decide whether PC loads the A-register's value (jump taken) or increments (no jump).
  - (c) `addressM` is always the A-register's low 15 bits; `pc` is the PC's current value (for instruction fetch next cycle).
2. `Memory(in[16],load,address[15]) → out[16]` — route addresses 0--16383 to `RAM16K` (Lab 3), 16384--24575 to `Screen`, and 24576 to `Keyboard`, using the address's top bits to select the region (a `DMux/Mux` pair on `address[14]`, then a further split on `address[13]` for `Screen` vs. `Keyboard` within the upper half — the exact split follows the memory map's address ranges).

3. **Computer(reset)** — connect a ROM32K (holding the program, loaded externally, read-only from the CPU's perspective), the CPU, and Memory: ROM's output feeds the CPU's **instruction** input at the PC's address; the CPU's **outM/writeM/addressM** drive Memory; Memory's output feeds back to the CPU's **inM**.
4. Run `CPU.tst`, `Memory.tst` and `Computer.tst` in the Hardware Simulator; then load `Mult.asm` and `Fill.asm` from Lab 4 onto the built **Computer** and confirm they still run correctly on your own CPU.
5. In the journal, draw the full datapath diagram of your CPU showing every wire between the ALU, registers, PC and the instruction decoder.

## 5.5 Figures

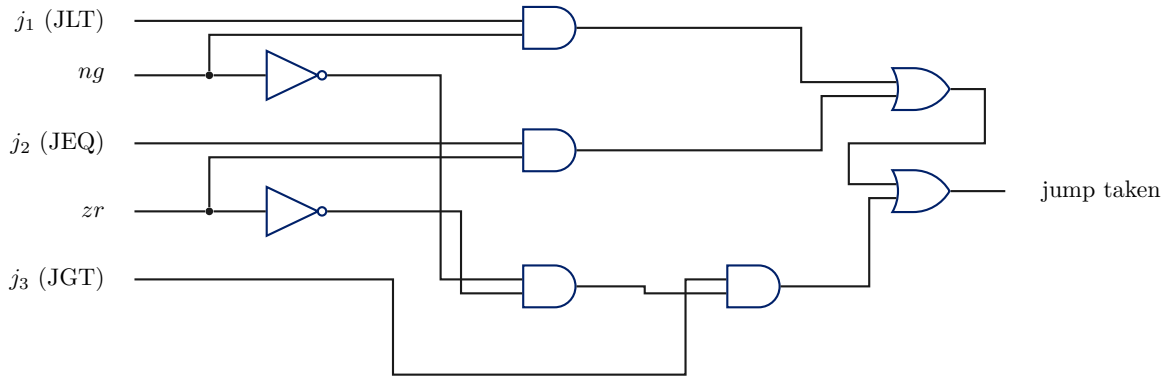


Figure 1: The C-instruction's jump logic at gate level. Three jump bits ( $j_1=\text{JLT}$ ,  $j_2=\text{JEQ}$ ,  $j_3=\text{JGT}$ ) combine with the ALU's  $ng/zr$  flags:  $jump = (j_1 \cdot ng) + (j_2 \cdot zr) + (j_3 \cdot \overline{ng} \cdot \overline{zr})$ . When *jump taken*, the PC loads the A-register value instead of incrementing.

| C-instruction field                              | bits   | Control effect                                                                  |
|--------------------------------------------------|--------|---------------------------------------------------------------------------------|
| <code>instruction[15..13]</code>                 | 111    | opcode: this is a C-instruction                                                 |
| <code>instruction[12]</code> ( <i>a</i> )        | 0/1    | ALU <i>y</i> input $\leftarrow$ A-register / <code>inM</code>                   |
| <code>instruction[11..6]</code> ( <i>ccccc</i> ) | 6 bits | $\leftarrow$ <code>zx,nx,zy,ny,f,no</code> (ALU control, Lab 2)                 |
| <code>instruction[5..3]</code> ( <i>ddd</i> )    | 3 bits | <b>dest</b> : load A ( $d_1$ ), load D ( $d_2$ ), <code>writeM</code> ( $d_3$ ) |
| <code>instruction[2..0]</code> ( <i>jjj</i> )    | 3 bits | <b>jump</b> : JLT/JEQ/JGT $\rightarrow$ PC load (Figure 1)                      |

Table 1: C-instruction decode: each bit field maps to one control signal, hardwired in `CPU.hdl`'s decode logic (no microcode, no sequence counter).

## 5.6 Observation / Journal Table

**Journal instruction (separate from the table above):** the full CPU datapath diagram (every wire between ALU, registers, PC and the instruction decoder — Figure 2 at block level, redrawn fully wired). This is the syllabus's own Deliverable 3.

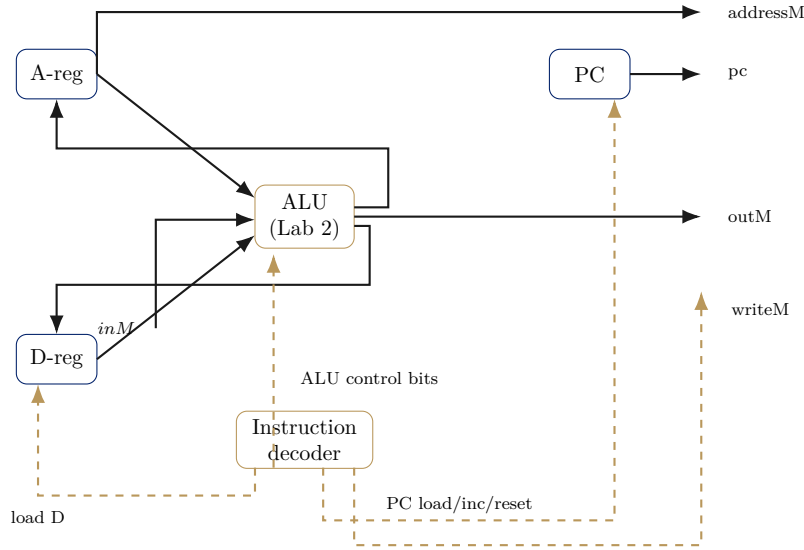


Figure 2: The Hack CPU datapath (block level). Solid lines are data: A and D feed the ALU, the ALU result goes to `outM` and loads back into A/D, A drives `addressM`, PC drives `pc`. Dashed lines are the decoder’s control signals (ALU bits, register load lines, PC load/inc/reset, `writeM`). This is the diagram to redraw, fully wired, for the journal deliverable.

| Chip / demo                  | Notes | Pass / Fail |
|------------------------------|-------|-------------|
| CPU (CPU.tst)                |       |             |
| Memory (Memory.tst)          |       |             |
| Computer (Computer.tst)      |       |             |
| Mult.asm on own Computer.hdl |       |             |
| Fill.asm on own Computer.hdl |       |             |

Table 2: Lab 5 observation table.

## 5.7 Viva-Voce Questions

1. Trace one full fetch–decode–execute cycle for a given instruction (an instructor-supplied example will be given at the viva): which chip fetches it, which decodes it, which executes it, and in what order do the register/PC updates happen?
2. Why does the A-instruction case bypass the ALU entirely, while the C-instruction case routes through it?
3. Explain how `jump` bits combine with the ALU’s `zr/ng` outputs to decide whether the PC loads or increments. Give one `jump` pattern and walk through the decision.
4. Why does `Memory.hdl` need address-range routing logic at all, rather than just wiring the CPU straight to `RAM16K`?
5. What breaks first if `ROM32K` were made writable by the CPU (i.e., programs could self-modify)? Why does the real Hack architecture keep it read-only?

## 5.8 Rubric

Reproduced verbatim from syllabi/CS401\_Lab\_Syllabus\_Nand2Tetris.pdf.

| Criterion                                                                   | Marks     |
|-----------------------------------------------------------------------------|-----------|
| CPU.hdl passes CPU.tst                                                      | 3         |
| Memory.hdl and Computer.hdl pass their test scripts                         | 3         |
| Mult.asm / Fill.asm run correctly on the built Computer                     | 2         |
| Datapath diagram complete and correct                                       | 1         |
| Viva: can trace one full fetch–decode–execute cycle for a given instruction | 1         |
| <b>Total</b>                                                                | <b>10</b> |